

0- Introdução

Para iniciar a análise/ estudo da biblioteca flower eu irei usar como base a simulação do o link [2023 Tutorial: FL Simulation with Flower | Introduction](#) . Para guiar a análise do flwr (nome usado para importar a biblioteca dentro do bloco de código) iremos analisar o bloco main e permear as utilizações que ele faz dos outros blocos

1- Imports

```
import hydra
from omegaconf import DictConfig, OmegaConf
from client import generate_client_fn
import flwr as fl
from dataset import prepare_dataset
from server import get_on_fit_config, get_evaluate_fn
```

Hydra → é um biblioteca pedida pelo tutorial para ajuda o flower a organizar a rede

Omegaconf → biblioteca usada para ajudar na elaboração e extração de dicionários em moldes diferentes do convencional (como o dicionário de estado do modelo)

client → importa a função de gerar cliente

flwr → importa o flower

dataset → bloco onde fazemos a partição dos dados e preparação dos mesmos

server → importamos as funções de pegar as configurações gerais “get_on_fit_config” e a função de avaliação do modelo global

2-Definição das configurações do servidor

```
@hydra.main(config_path="conf",config_name="base",version_base=None)
def main(cfg:DictConfig):

    print(OmegaConf.to_yaml(cfg))
```

pega e printa (a dito de exemplificação) as configurações gerais do servidor

```
! base.yaml X
D: > PROGRAMAÇÃO > ML > FL > Flower Tutorial > conf > ! base.yaml
1  ---
2
3  num_rounds: 10
4  num_clients: 100
5  batch_size : 20
6  num_clients_per_round_fit : 10
7  num_clients_per_round_eval : 25
8  num_classes: 10
9  config_fit :
10 |   lr: 0.01
11 |   momentum: 0.9
12 |   local_epochs : 1
13
```

- número de rodadas
- número de clientes
- tamanho do pacotes de dados na divisão
- número de clientes por rounds de ajuste
- número de cliente por round de avaliação
- número de classes (para o dataset do MNIST usamos 10)
- e as configurações de ajuste geral que são passadas a cada rodada

3- Prepara o dataset

```
trainloaders, validationloaders, testloader = prepare_dataset(cfg.num_clients,  
                                                             cfg.batch_size)  
  
print(len(trainloaders), len(trainloaders[0].dataset))
```

Chama a função do bloco de código `dataset.py` para dividir e preparar o dataset em dados de treino, avaliação e teste, para o numero de clientes e o número do batch.

```

from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor, Normalize, Compose
from torch.utils.data import random_split, DataLoader
import torch

def get_MNIST(data_path:str = './data'):
    tr = Compose([ToTensor(), Normalize((0.1307,), (0.3081,))])
    trainset = MNIST(data_path, train=True, download=True, transform=tr)
    testset = MNIST(data_path, train=False, download=True, transform=tr)

    return trainset, testset

def prepare_dataset(num_partitions : int ,
                    batch_size: int,
                    val_ration : float = 0.1):

    trainset, testset=get_MNIST()

    #split trainset into 'num_partitions' trainsets

    num_images = len(trainset)// num_partitions
    partiton_len = (num_images) * num_partitions
    num_images_per_partition = len(trainset) // num_partitions
    partition_sizes = [num_images_per_partition] * num_partitions
    remainder = len(trainset) % num_partitions
    partition_sizes[-1] += remainder
    gen = torch.Generator().manual_seed(2023)
    trainsets = random_split(trainset, partition_sizes, generator=gen)
    trainloaders=[]
    valloaders=[]
    for trainset_ in trainsets:
        num_total = len(trainset_)
        num_val = int(val_ration*num_total)
        num_train = num_total - num_val

        for_train, for_val =random_split(trainset_, [num_train,num_val], generator=gen)

        trainloaders.append(DataLoader(for_train, batch_size=batch_size, shuffle=True, num_workers = 2))
        valloaders.append(DataLoader(for_val, batch_size=batch_size, shuffle=False, num_workers = 2))

    testloader= DataLoader(testset,batch_size=128)

    return trainloaders, valloaders, testloader

```

Sem se ater muito ao código ele basicamente normaliza, embaralha e divide em pacotes para os clientes.

4- Cria os clientes

```
def generate_client_fn(trainloaders, validationloaders, cfg.num_classes):  
    client_fn = generate_client_fn(trainloaders, validationloaders, cfg.num_classes)
```

Usando a função importada do bloco de código `client.py` ele cria os clientes com os seus respectivos conjuntos de dados de treino, validação e o número de classes definido no arquivo de configurações `base.yaml`

```
def generate_client_fn(trainloader, valloader, num_classes):  
    def client_fn(cid: str):  
        return FlowerClient(trainloader=trainloader[int(cid)],  
                             valloader=valloader[int(cid)],  
                             num_classes=num_classes).to_client()  
    return client_fn
```

Função de criação do cliente que basicamente instancia um objeto da classe `FlowerClient`, classe criada no bloco de [client.py](#)

```
class FlowerClient(fl.client.NumPyClient):  
    def __init__(self,  
                 trainloader,  
                 valloader,  
                 num_classes) -> None:  
        super().__init__()  
        self.trainloader = trainloader  
        self.valloader = valloader  
        self.model = Net(num_classes)  
        self.device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

Define os atributos do cliente:

- dados de treino
- dados de validação

- modelo
- serviço

Os últimos dois atributos são atributos definidos pelo pytorch sendo o primeiro o algoritmo do treino do cliente, e o segundo sendo onde o treino irá ocorrer, sendo cpu ou gpu

```
def set_parameters(self, parameters):  
    params_dict = zip(self.model.state_dict().keys(), parameters)  
    state_dict = OrderedDict({k: torch.tensor(v) for k, v in params_dict})  
    self.model.load_state_dict(state_dict, strict=True)
```

Seta os parâmetros, vindos do servidor, que vem como um numpy array e converte para torch.tensor para que o modelo do pytorch possa utilizar

```
def get_parameters(self, config: Dict[str, Scalar]):  
    return [val.cpu().numpy() for _, val in self.model.state_dict().items()]
```

Retorna os parâmetros para a rede convertendo os valores como numpy array

```
def fit(self, parameters, config):

    self.set_parameters(parameters)

    lr = config['lr']
    momentum = config['momentum']
    epochs = config['local_epochs']

    optim = torch.optim.SGD(self.model.parameters(), lr=lr, momentum=momentum)
```

Pega os parâmetros e seta eles a cada rodada, atualiza outras configurações dada pelo arquivo de configurações e otimiza o modelo, a técnica de otimização utilizada no caso é o Stochastic Gradient Descent (SGD)

```
train(self.model, self.trainloader, optim, self.device)

return self.get_parameters({}), len(self.trainloader), {}
```

Chama a função treino e retorna os parâmetros e o tamanho do conjunto de dados de treino, pois estamos usando o FedAvg, que vamos ver mais pra frente em estratégia, logo, precisamos enviar o tamanho do dataset

A função train e test serão vistas quando dermos uma rápida passada pelo bloco de [model.py](#)

```
def evaluate(self, parameters: NDArrays, config: Dict[str, Scalar] ):
    self.set_parameters(parameters)

    loss, accuracy = test(self.model, self.valloader, self.device)

    return float(loss), len(self.valloader), {'accuracy': accuracy}
```

Avalia o modelo depois de setar os parâmetros recebidos e retorna o erro e acuracidade do modelo depois de rodar um teste

5- Estratégia da rede

```
strategy = fl.server.strategy.FedAvg(fraction_fit=0.00001,
                                     min_fit_clients=cfg.num_clients_per_round_fit, #clientes mexidos por rodada
                                     fraction_evaluate = 0.00001,
                                     min_evaluate_clients = cfg.num_clients_per_round_eval, #clientes avaliados por rodada
                                     min_available_clients=cfg.num_clients,
                                     on_fit_config_fn=get_on_fit_config(cfg.config_fit), #pega os parametros de ajuste dos clientes
                                     evaluate_fn=get_evaluate_fn(cfg.num_classes, #avalia o modelo global
                                                                testloader))
```

Variável que carrega as configurações, atribuições e estratégias do treino da rede, no caso utilizamos FedAvg. Cabe olharmos para as funções do servidor `get_on_fit_config` e `get_evaluate_fn`:

```
def get_on_fit_config(config: DictConfig):
    def fit_config_fn(server_round: int):

        return {'lr': config.lr, 'momentum': config.momentum, 'local_epochs': config.local_epochs}

    return fit_config_fn
```

Pega as configurações de ajuste para cada rodada


```
def get_evaluate_fn(num_classes, testloader):
    def evaluate_fn(server_rounds: int, parameters, config):

        model = Net(num_classes)
        model.to(device)

        device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

        params_dict = zip(model.state_dict().keys(), parameters)

        state_dict = OrderedDict({k: torch.tensor(v) for k, v in params_dict})

        model.load_state_dict(state_dict, strict=True)

        loss, accuracy = test(model, testloader, device)

        return loss, {'accuracy': accuracy}

    return evaluate_fn
```

A avaliação dos modelos locais que o servidor faz é feita pegando o modelo da rede, carregando e normalizando o dicionário de estado, carregando o estado e testando o modelo.

Basicamente o que a mesmas funções feitas no cliente, a diferença é que na rede ele vai avaliar o modelo global construído pelos parâmetros agregados pelos nós locais e o cliente ele vai avaliar o modelo local.

6- Inicia a simulação

```
history = fl.simulation.start_simulation(client_fn=client_fn, #inicia o cliente
                                       num_clients=cfg.num_clients, #inicia N clientes
                                       config=fl.server.ServerConfig(num_rounds=cfg.num_rounds), #inicia o servidor por um num rounds
                                       strategy=strategy) #modela a estratégia do servidor
```

A função de iniciar a simulação que inicia e envia as configurações dos clientes e da rede, como também guardar todos os dados de perda e acurácia de cada round.

